

当前 Agent 实际执行问题复盘（KDD Cup 2026）

本文基于 `note.md`、`artifacts/runs` 中的实际运行结果，以及 `src/data_agent_baseline` 源码整理。结论是：当前失败不是单个题目或单条 prompt 的问题，而是“上下文过载、语义绑定不收敛、最终答案必须由 LLM 逐行吐出、验证层偏弱”叠加后的系统性问题。

当前已增加的两个上下文能力改进

在 Phase1 代码基础上，当前版本已经补上了两类对 Phase2 更关键的上下文处理能力：PDF 文档理解入口和视频证据入口。它们的共同效果是：把原本不容易被 Agent 稳定使用的非结构化材料，先转换成更适合检索、阅读和多步推理的上下文资产，再交给 Agent 使用。

PDF 转 Markdown 的效果：

- PDF 不再只是一个难以直接读取的附件，而是会变成可搜索、可阅读的 Markdown 文档。
- Agent 可以像处理普通 `.md` 文档一样读取 PDF 内容、搜索关键词、定位章节和抽取文本证据。
- 带目录的 PDF 会保留更清晰的章节层级，减少模型在长文档中盲读和误定位的概率。
- 中文或英文文档中常见的硬换行会被合并，文本连续性更好，更利于规则、定义、业务说明和报告内容的抽取。
- 如果任务中原本已经存在同名 Markdown，PDF 转换结果会以区分后的新文档出现，避免覆盖已有材料。

视频处理的效果：

- 视频不再依赖模型直接处理原始视频文件，而是被转换成“时间线文本 + 稳定画面 + 语音转写”的组合证据。
- Agent 能看到视频中的关键静态画面，尤其适合页面录屏、报表讲解、幻灯片演示和数据看板类视频。
- 视频中的语音会被转成带时间戳的 transcript，使 Agent 能利用口头说明、筛选口径、任务条件和对话信息。
- 画面与语音会按时间片段组织到同一份 timeline 中，便于模型把“此时看到什么”和“此时说了什么”对应起来。
- 原始视频不会完整塞入首轮上下文，降低多模态请求体和 token 压力，也避免模型只凭单帧画面忽略音频线索。
- 如果视频处理失败，系统仍会给出失败说明文档，Agent 可以继续利用其他上下文完成任务，而不是整题直接中断。

这两个改进已经显著扩展了 Agent 可处理的数据形态：Phase1 主要面向表格、数据库和文本文件；当前版本开始能够把 PDF 报告和视频材料纳入统一上下文。但从实际执行结果看，它们解决的是“材料可进入 Agent”的问题，还没有完全解决“Agent 必然正确使用这些材料”的问题。后文的问题复盘主要聚焦在这些新增上下文进入系统之后，仍然暴露出的执行链路缺陷。

第一次执行效果

主分与基础指标

指标	值
任务总数	60

指标	值
生成 prediction.csv 的任务数	60
Primary Score ($\lambda=0.1$)	0.3674
Mean Recall	0.3750
Mean Redundancy Rate	0.6597

按难度拆分表现

难度	任务数	有预测	完全正确题数	Primary($\lambda=0.1$)	Mean Recall	Mean Redundancy
unknown	60	60	20	0.3674	0.3750	0.6597

运行时摘要

指标	值
可用耗时任务数	60
总时长（分钟）	53.19
平均耗时（秒）	189.297
中位耗时（秒）	122.523
P95 耗时（秒）	510.441
最长耗时（秒）	1617.748
可用模型轮数任务数	60
平均模型轮数	8.72
最大模型轮数	36

当前的主要问题是：执行的太慢了，尽管平均模型轮数并不大，但是由于首轮上下文就已经过长，导致推理很慢，非常消耗token，执行一次API花费近60元。

当前比较明显的问题

1. 首轮上下文过载，数据目录和视频帧一起挤占推理空间

note.md 里记录的首轮输入已经达到约 131k tokens，其中 **Global Data Profile** 约 114k tokens，占 87% 以上，稳定帧图片约 14k tokens，占 10% 以上。实际 trace 也能看到多个任务的 first human message 达到 18 万到 41 万字符。

影响是：模型在真正解题前已经被大量低相关 catalog、字段枚举、图片 token 占满。对于 Phase 2 的“一个任务一个视频 + 大量表格/文档”的设置，这会明显增加误选字段、忽略关键音频、以及模型请求失败或降级的概率。

task_1为例：

组成部分	字符数/数量	实际 Token 数	占总 Token 比例	说明
数据画像 (<code>Global Data Profile</code>)	373,391 字符	114,268	87.19%	数据库所有表的 Schema、字段类型、非重复值 Top N、极值及 Join 关系等。
视频稳定帧图片 (13 张图像)	13 张 JPEG 图片	14,197	10.83%	视频预处理生成的 13 个稳定帧以 <code>image_url</code> 传入多模态模型，平均每张图消耗 1,092 Tokens。
系统提示词 (<code>System Prompt</code>)	9,137 字符	1,869	1.43%	Agent 运行的全局 System Instruction、工具使用规则等。
视频预处理上下文文本	-	576	0.44%	指明 timeline.md 及各稳定帧文件路径的辅助文本说明。
用户问题 (<code>User Query</code>)	-	33	0.03%	用户的原始提问文本（含包裹标签）。
其它引导词 & Action Trigger	-	118	0.08%	上下文注入的前言 Preamble (59) 和提示首步操作的 Action Trigger (35) 等。
协议与聊天模板开销 (Chat Overhead)	-	~100	0.08%	消息格式包裹符及多模态序列化封装等额外消耗。
总计	-	131,061	100.00%	第一次 LLM 请求的全部 Input Tokens

2. 最终答案提交层过于脆弱：所有行都必须由 LLM 生成 JSON

当前 `answer` 工具只接受：

- `columns: list[str]`
- `rows: list[list[Any]]`

这意味着最终提交没有“文件引用”“SQL 结果引用”“Python 导出路径”这种确定性通道。哪怕工具已经算出了 137 行、294 行、12962 行，最后仍然要模型在 tool-call JSON 里重新生成完整表。这个设计直接解释了几个实际失败：

- `task_4`：查询阶段能找到 137 行，但一次运行只提交了 64 行，且停在 `国家股` 附近；后续运行虽然提交 137 行，却多输出了非 gold 需要的列。
- `task_13`：官方答案约 12962 行，模型发现了总行数后仍然只提交 10 行样例/代表行，因为让 LLM 吐出上万行本身就不可靠。
- `task_10`：官方答案 294 行单列，模型最终提交了 20 行，还保留了辅助列 `EndDate`。

这类问题不是简单要求模型“不要截断”就能解决的；提交层需要支持由工具产生最终 CSV，并由 runner 按引用写出。

3. 视频证据链没有变成强制步骤，模型容易被稳定帧带偏

Phase 2 的视频包含中英文人机对话和页面/幻灯片信息。当前预处理确实会做稳定帧提取和 ASR 时间线：

- `run/context_preprocessor.py` 会把 video 转成 generated timeline 和 stable frames。

- `run/video_preprocessor.py` 会提取 stable frames、转写 transcript，并已有 `repair_transcript_mojibake()` 修复常见乱码。
- `agents/multimodal.py` 明确不直接附 raw video，只附稳定帧和文本提示。

问题是，稳定帧被前置塞进初始消息，而音频/时间线仍然依赖模型主动 `read_doc`。如果模型被图片中的表格/页面状态先入为主误导，或者没有充分读取 transcript，就会丢掉关键口径。

`task_6` 是最典型例子。note 中记录了早期 `read_doc(video/briefing_timeline.md)` 出现乱码，导致模型丢失关键音频线索。当前代码已有 mojibake repair，但系统层仍没有强制“视频题必须先检索/读取 transcript，再结合图片验证”。实际 gold 有 23 行，模型只输出 4 行，并且使用全称列，说明它仍被局部画面和不完整口径带偏。

4. 文档/Markdown 正则抽取缺少结构化支持(老问题)

当前文档工具主要是：

- `read_doc_preview()`：按 UTF-8 读取文本，可按 heading 取片段。
- `search_doc_text()`：做关键词/正则搜索并分页返回命中。

这些工具适合找证据，但不适合稳定地从长 markdown 中抽取大量重复结构。代码里虽然存在 `tools/memagent_etl.py`，可把文档抽成 SQLite，但默认 registry 没有注册相关工具；而且历史说明中也提到该方向不稳定、较慢。

这解释了 `task_15`：它需要从 markdown 中做规则化/正则化抽取，模型能靠搜索和 Python 找到一部分，但最后只输出 24 行，gold 约 50 行，并且列名多了 `record`、大小写不匹配。

针对前三个问题的优化方案

前三个问题可以归纳为同一个系统方向：当前 Agent 仍然偏向“首轮一次性塞入大量材料，最后再由模型一次性吐出答案”。优化目标应改成“轻量首轮定位，按需读取证据，工具确定性产出答案，视频任务强制走完整证据链”。

1. 首轮上下文过载：改成分层上下文

当前最大浪费来自完整 `Global Data Profile` 和首轮自动附加稳定帧。优化目标是把首轮输入控制到一个稳定预算内，例如 10k-20k tokens，而不是让 task_1 这类任务一开始就达到 131k tokens。

建议方案：

- 首轮只给模型最小必要信息：用户问题、文件树、表/文件名称、关键路径、视频 timeline 路径。
- 不再默认注入完整字段枚举、Top N distinct values、全量 schema 画像。
- 将数据画像拆成两层：第一层是轻量 catalog，只说明有哪些表、哪些文件、大概字段名；第二层由模型按需调用工具查看具体表结构、字段取值、文档片段。
- 视频稳定帧默认不在首轮全部附加，最多只附 0-2 张概览图；更多图片必须在读取 timeline 后按时间段选择。
- 对首轮上下文设置硬预算，超过预算时自动裁剪低价值部分，而不是继续堆入模型输入。

预期效果：

- API 成本和首轮延迟显著下降。
- 模型不再被无关字段枚举和图片 token 淹没。

- 减少误选字段、跳过文档、忽略音频线索的问题。

验收指标：

- task_1 首轮 tokens 从约 131k 降到 20k 以下。
- 平均模型调用耗时下降至少 40%。
- 单次 60 题运行成本明显下降。

2. 最终答案提交脆弱：改成工具产出答案

当前 `answer` 必须由 LLM 把所有行写进 JSON，这是 `task_4`、`task_10`、`task_13` 这类题失败的核心原因。优化目标是让 LLM 不再负责吐大表，只负责确认哪个工具结果是最终答案。

建议方案：

- 支持文件式提交：Python、SQL 或抽取工具生成最终 CSV 后，Agent 只提交这个文件路径。
- 支持最近一次工具结果提交：如果 SQL/Python 已经得到完整结果，可以直接把该结果提升为最终答案。
- 最终提交前增加轻量校验：列名是否符合题目要求、行数是否异常过少、是否存在明显多余辅助列、是否有重复行。
- 对大行数答案设置规则：超过一定行数时禁止 LLM 手写 rows，必须走工具产出。

预期效果：

- 大表题不再因为模型截断、漏行、JSON 过长而失败。
- `task_13` 这种上万行输出可以稳定提交。
- `task_4`、`task_10` 这类“工具已算出但最终提交错”的问题会明显减少。

验收指标：

- 大于 100 行的答案不再由 LLM 直接生成完整 JSON。
- `prediction.csv` 行数与工具计算结果一致。
- Mean Recall 提升，Mean Redundancy 降低。

3. 视频证据链不强制：改成视频任务固定流程

当前视频处理已经有 timeline、transcript 和稳定帧，但模型不一定会先读 timeline。优化目标是：只要任务含视频，就必须先读视频时间线，再结合图片和数据作答。

建议方案：

- 视频任务首步强制读取 timeline，不允许直接根据稳定帧或表格回答。
- 首轮提示中明确视频题必须先使用 transcript，再看稳定帧。
- 稳定帧从“首轮自动塞入”改成“读完 timeline 后按时间段选择”。
- 在最终提交前增加视频证据检查：是否读取过 timeline、是否使用过 transcript、是否检查过与答案相关的稳定帧。
- 如果没有形成视频证据链，则要求继续取证，不能直接提交最终答案。
- 对 ASR 失败或低质量 transcript，明确降级策略：使用更多稳定帧和其他上下文，但不能假装音频已经被理解。

预期效果：

- 减少模型被单张画面带偏。
- 提升视频中口头条件、筛选规则、对话信息的利用率。
- **task_6** 这类依赖音频口径的题会更稳定。

验收指标：

- 所有视频题 trace 中都能看到 timeline 被读取。
- 视频题最终答案前出现 transcript / 稳定帧联合证据。
- **task_6** 输出行数和字段口径明显改善。

推荐落地顺序

1. 先做首轮上下文瘦身，因为它直接降低成本和耗时。
2. 同时把视频稳定帧从“默认全塞”改成“读 timeline 后按需使用”。
3. 接着改最终答案提交方式，这是提升 Recall 和解决大表题的关键。
4. 最后增加提交前校验和视频证据检查，让流程更稳定。

一句话概括：先让模型少看无关东西，再让工具负责大规模确定性输出，最后强制视频题走完整证据链。